

本稿主要整理个人认为高概率会涉及的知识点，不保证好用…

Ss1：感觉上是纯引入章，个人未找到考点，知识点在其后部分应该均有涉及

1. 程序的转换过程 (ss7)

源程序(.c) → [预处理] → 修改后的源程序(.i) → [编译] → 汇编程序(.s) → [汇编] → 可重定位目标程序(.o) → [链接] → 可执行目标程序。

避坑：选择题常考“汇编阶段生成什么文件”，注意区分.s（汇编文本）和.o（机器码目标文件），通常汇编器输出的是.o。

2. 存储层次结构 (ss6)

寄存器 → L1/L2/L3 Cache(SRAM) → 主存(DRAM) → 本地磁盘 → 网络分布式文件系统。

规律：越往上速度越快、容量越小、每字节价格越贵。

3. 操作系统的核心抽象

进程（对 CPU 的抽象）、虚拟内存（对主存和磁盘的抽象）、文件（对 I/O 设备的抽象，Unix 中万物皆文件）。

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
long double	-	-	10/16
pointer	4	8	8

第 2 章 信息的表示和处理

1. 信息存储与数据类型

基本概念

进制转换：参考计系 1 即可

数据类型大小 (x86-64 vs IA-32):

字节顺序 (大端 vs 小端):

小端法：x86 默认。最低有效字节存放在低地址。（“小头在前”）

大端法：网络传输/Sun。最高有效字节存放在低地址。

这里还有一个双端，可以按需要配置。

数据:	12345678		
大端法地址		小端法地址	
0x103	78	0x103	12
0x102	56	0x102	34
0x101	34	0x101	56
0x100	12	0x100	78

例：int i = 0x86231132, &i = 0x400320。问地址 0x400322 上的字节是多少？

Ans: x86-64（我们考的）是小端：0x400320 存 32，21 存 11，22 存 23，23 存 86。答案是 0x23。

2. 整数表示与隐式转换

计系 1 未涉及的部分：

扩展与截断：

扩展 (短转长)：无符号零扩展，有符号符号扩展。

截断 (长转短)：直接砍掉高位，保留低位。int i = 0xabcdef01; short si = i; → si 是 0xef01。

C 语言隐式转换：有符号 + 无符号的情况，有符号会变成无符号

例：-1 < 0U → **False**。 -1 会变成 Umax，大于 0。

2147483647U > -2147483647 - 1 → **False**。 -2147483648 变成无符号后比 2147483647 大。

3. 整数运算与移位

移位运算

逻辑右移：补 0（无符号数）。

算术右移：补符号位（有符号数）。

注意：unsigned x=15; int y=-4; x>>2; y>>2; → x=3, y=-1。

编译器优化 (乘除法)

乘法变移位：x * 14 → (x<<3) + (x<<2) + (x<<1)。或者 x*30 → (x<<5) - (x<<1)。

除法变移位：x / 4 → x >> 2。

注意：负数除法会向零取整（如 -7/4 = -1），算术右移向下取整（-7>>2 = -2）。

所以负数右移前通常要加偏置（如 (x+3)>>2）。

4. 浮点数 (IEEE 754) 参考一下计系 1

基本结构

实际值 $V = (-1)^s \times M \times 2^E$

单精度 (32 位): 1 位 s + 8 位 exp + 23 位 $frac$ 。

双精度 (64 位): 1 位 s + 11 位 exp + 52 位 $frac$ 。

三种状态

规格化: exp 不全 0 不全 1。 $E = e - Bias$ (单精度 127)。 $M = 1 + f$ (隐含最高位 1)。

非规格化: exp 全 0。 $E = 1 - Bias$ (单精度 -126)。 $M = 0 + f$ (隐含最高位 0)。用于表示 0 和极靠近 0 的数。

特殊值: exp 全 1。 $frac = 0$ 为 ∞ , $frac \neq 0$ 为 NaN。

自定义浮点数解码

例: 8 位浮点数 (阶码 4 位, 小数 3 位, 无符号位) “0 0110 110”表示的数值。

解: 1.看阶码: 0110 (6)。不全 0 不全 1 $\rightarrow$ 规格化。

2.算 E : $Bias = 2^{4-1} - 1 = 7$ 。 $E = 6 - 7 = -1$ 。

3.看尾数: 110。 $M = 1 + f = 1 + (1/2 + 1/4) = 1.75$ 。

4.算结果: $V = 1.75 \times 2^{-1} = 0.875$ 。

舍入规则

默认向偶数舍入

正好一半(尾 1000...): 向偶数靠拢 (让保留的最低位变成 0)。

如 1.5 $\rightarrow$ 2, 2.5 $\rightarrow$ 2。

浮点数性质 (精度导致)

不满足结合律: $(A + B) + C \neq A + (B + C)$ 。大数吃小数, 精度丢失。

不满足分配律: $A \times (B + C) \neq A \times B + A \times C$ 。舍入误差。

第三章：程序的机器级表示（汇编与 C 语言互译）

1. 常用汇编指令

这里只列考试和逆向中高频的指令，按功能分类。

数据传送与地址计算

mov Src, Dst: 数据传送。源在前，目的在后。不能内存到内存直接传。

leaq Src, Dst: 加载有效地址。不访问内存，只算地址。常用来做算术运算，比如

leaq (%rdi,%rdi,4), %rax 相当于 $rax = 5 * rdi$ 。**不改变条件码。**

movsbl Src, Dst: 符号扩展传送（如 1 字节 扩展到 4 字节）。

movzbl Src, Dst: 零扩展传送。

算术与逻辑运算

add / sub / imul Src, Dst: 加、减、乘。 $Dst = Dst \text{ op } Src$ 。

sal / shl (左移), sar / shr (右移): **sar** 是算术右移（补符号位），**shr** 是逻辑右移（补 0）。移位量可以是立即数或 `%cl` 寄存器。

and / or / xor Src, Dst: 按位与、或、异或。

test Src, Dst: 按位与，但不保存结果，只设置条件码。常用 **testq %rax, %rax**，用来判断 `%rax` 是等于 0、大于 0 还是小于 0。

条件码与跳转

cmp Src, Dst: 做减法 $Dst - Src$ ，不保存结果，只设置条件码。

jcc Label: 条件跳转。cc 代表条件，如 **je** (等于), **jne** (不等于), **jg** (大于), **jl** (小于), **jge** (大于等于)。

cmovcc Src, Dst: 条件传送。如果条件满足，把 `Src` 传给 `Dst`。对应 C 语言的三目运算符 `?:`。

关于内存寻址：`i (%R1, %R2, j)` 表示地址 $R1 + j * R2 + i$

2. 常见程序结构的汇编模式

条件分支 (if-else) 编译器通常有两种实现方式：

条件跳转: 先算 `then` 和 `else` 的结果，用 `cmp + jcc` 跳转到对应的代码块。

条件传送: 把 `then` 和 `else` 的结果都算出来，用 `cmp + cmovcc` 选择其中一个。这种方式没有分支，性能更好，但要求两个结果都能安全计算（不能有除零或内存越界）。

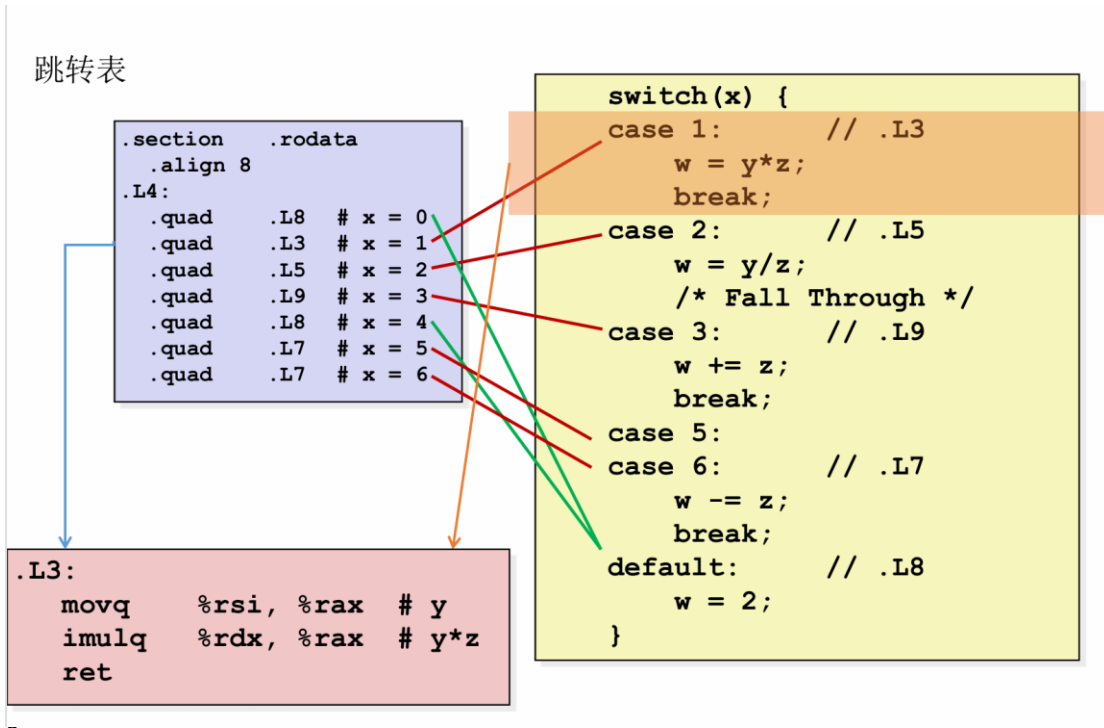
循环结构 (for / while / do-while) 编译器通常会把 `for` 和 `while` 优化成 `do-while` 的形式以减少一次跳转。步骤：

初始化: 在进入循环前，给循环变量和累加器赋初值（如 `movl $0, %eax`）。

条件判断: 在循环体前或后，用 `test` 或 `cmp` 比较循环变量和边界，用 `jle / jge` 等跳出循环。

循环体与更新: 执行计算，然后更新循环变量（如 `addq $1, %rcx`），最后无条件跳回判断处或直接判断。

Switch: 直接看 PPT 吧…对应来说就是：利用 x 的值运算得到对应 case 在跳转表中的位置，然后跳过去



一个简单的循环样例：

```
1 int main()
2 {
3     int i=0;
4     int sum=0;
5     while(i<10)
6     {
7         sum+=i;
8         i++;
9     }
10    return 0;
11 }
```

```
main:
    push    rbp
    mov     rbp, rsp
    mov     DWORD PTR [rbp-4], 0
    mov     DWORD PTR [rbp-8], 0
    jmp     .L2
.L3:
    mov     eax, DWORD PTR [rbp-4]
    add     DWORD PTR [rbp-8], eax
    add     DWORD PTR [rbp-4], 1
    jmp     .L2
.L2:
    cmp     DWORD PTR [rbp-4], 9
    jle     .L3
    mov     eax, 0
    pop     rbp
    ret
```

(左 C，右汇编)

一些其他知识：

1. 数组（一维与多维）

内存布局：连续分配。

访问方式：基址 + 索引 × 元素大小。

汇编体现：利用比例因子寻址。比如访问 `int A[i]`，汇编通常是 `movl (%rdi, %rsi, 4), %eax`（`%rdi` 是基址，`%rsi` 是索引 `i`，4 是 `int` 的字节数）。

多维数组：按“行优先”存储。`A[i][j]` 的偏移量是 $(i * \text{列数} + j) * \text{元素大小}$ 。编译器会用 `lea` 指令或移位加法快速算出总偏移，然后一次性访存。

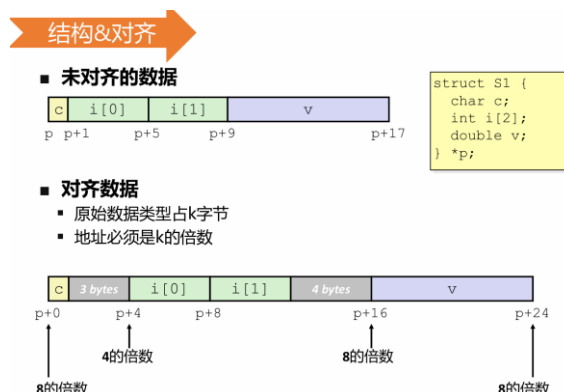
表达式	类型	值	汇编代码
E	int *	X_E	<code>movl %edx, %eax</code>
E[0]	int	$M[X_E]$	<code>movl (%edx), %eax</code>
E[i]	int	$M[X_E + 4i]$	<code>movl (%edx, %ecx, 4), %eax</code>
&E[2]	int *	$X_E + 8$	<code>leal 8(%edx), %eax</code>
E+i-1	int *	$X_E + 4i - 4$	<code>leal -4(%edx, %ecx, 4), %eax</code>
*(E+i-3)	int	$M[X_E + 4i - 12]$	<code>movl -12(%edx, %ecx, 4), %eax</code>
&E[i]-E	int	i	<code>movl %ecx, %eax</code>

2. 结构体

内存布局：按字段声明顺序排列，但为了满足**内存对齐**（比如 8 字节 double 的地址必须是 8 的倍数），编译器会在中间或末尾插入空隙。

访问方式：基址 + 固定常数偏移。

汇编体现：编译器在编译阶段就算死了每个字段的偏移量。比如访问结构体指针 p 的某个字段，汇编直接是 `movl 12(%rdi), %eax`（12 就是该字段硬编码的偏移量，不需要运行时计算）。



3. 联合体

内存布局：所有成员重叠共享同一块内存，总大小等于“最大成员”的大小。

访问方式：和结构体一样是“基址+偏移”，但因为内存重叠，它常被用来以不同的数据类型强行解释同一块内存。比如把一个 float 放进 union，再按 unsigned int 读出来，就能直接看到浮点数的底层 IEEE 754 位模式，完全 bypass 掉浮点转换指令。

4. 嵌套数组 vs 指针数组

嵌套数组（如 `int A[3][5]`）：一整块连续内存。访问 `A[i][j]` 只需要算一次数学偏移，一次访存。

指针数组（如 `int *A[3]`）：数组里存的是指针。访问 `A[i][j]` 需要**两次访存**：先根据 i 算出偏移，从内存读出指针（拿到真正的基址），再根据 j 算出偏移，去读真正的数据。汇编里会看到两次带括号的内存读取操作（如 `Mem[Mem[Base + i*8] + j*4]`）。

栈帧、过程调用与缓冲区溢出

1. 栈帧布局与寄存器约定

参数传递：前 6 个整数/指针参数用 `%rdi, %rsi, %rdx, %rcx, %r8, %r9`。返回值整数在 `%rax`。

寄存器保存约定：

调用者保存：`%rax, %rcx, %rdx, %rsi, %rdi, %r8-%r11`。被调用函数可以随意修改，调用者如果要保留值必须自己先压栈。

被调用者保存：`%rbx, %rbp, %r12-%r15`。被调用函数如果要使用，必须先在栈上保存旧值，退出前恢复。

栈帧布局（从高地址到低地址）：

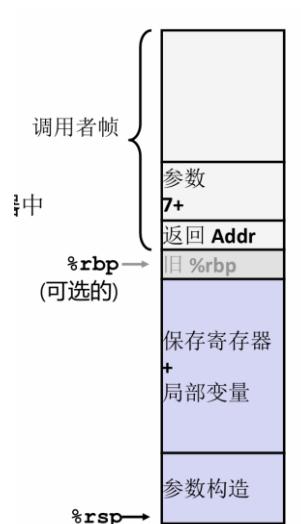
调用者的参数（如果超过 6 个）

返回地址（`call` 指令压入）

旧 `%rbp`（可选）

被调用者保存的寄存器（如 `%rbx`）

局部变量（包括数组、金丝雀等）

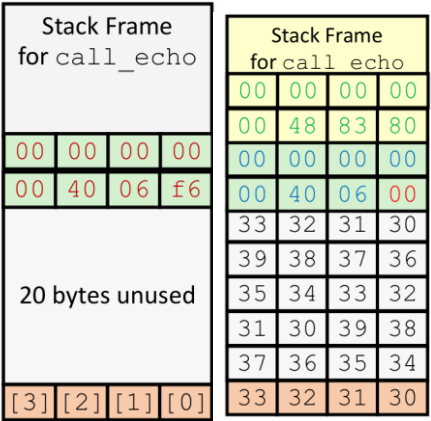


2. 局部数组在栈上的初始化

规则：x86-64 默认小端法，低地址存低字节，高地址存高字节。

例题：给 `int arr[4] = {2, 0, 1, 9}`，补全汇编里的 `movl`。

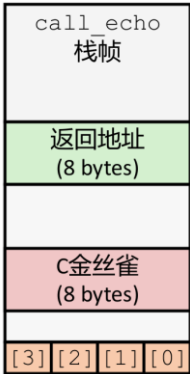
`int` 是 4 字节。`arr[0]` 在低地址，`arr[3]` 在高地址。
假设栈空间分配了 16 字节 (`subq $0x10, %rsp`)，基址是 `%rsp`。
`arr[0]` 存在 `(%rsp)`，值为 2。
`arr[1]` 存在 `4(%rsp)`，值为 0。
`arr[2]` 存在 `8(%rsp)`，值为 1。
`arr[3]` 存在 `12(%rsp)`，值为 9。
这里注意立即数的写法，如 `movl $0x09, 12(%rsp)`。



3. 缓冲区溢出与栈保护

溢出原理：向栈上的局部数组（如 `char buf[8]`）写入超过其大小的数据，会覆盖栈帧中更高地址的数据（如返回地址）。

栈保护机制（金丝雀）：编译器在局部变量和返回地址之间插入一个随机值。
函数入口：从 `%fs:40` 取出金丝雀值，存入栈中（如 `movq %fs:40, %rax -> movq %rax, 8(%rsp)`）。
函数出口：检查栈中的金丝雀是否被修改（`xorq %fs:40, %rax -> je .L9`）。如果被修改，说明发生了溢出，调用 `__stack_chk_fail` 终止程序。



这里做题时画出栈结构会比较有帮助

一些补缺：

第二章：移位运算的优先级陷阱：考过 $x \ll 4 + x \ll 3$ ，因为 $+$ 的优先级高于 $\ll$ ，所以必须加括号写成 $(x \ll 4) + (x \ll 3)$ ，否则直接报错或逻辑错误。

浮点数舍入细节：默认“向偶数舍入”，正好一半时（如尾数是 1000...），要让保留的最低位变成 0（比如 1.5 变成 2，2.5 也变成 2）。

第三章：条件码（CF/ZF/SF/OF）

ZF（零标志）：结果为 0 时置 1。

SF（符号标志）：结果为负时置 1。

OF（溢出标志）：有符号数溢出时置 1。

CF（进位标志）：无符号数溢出时置 1。

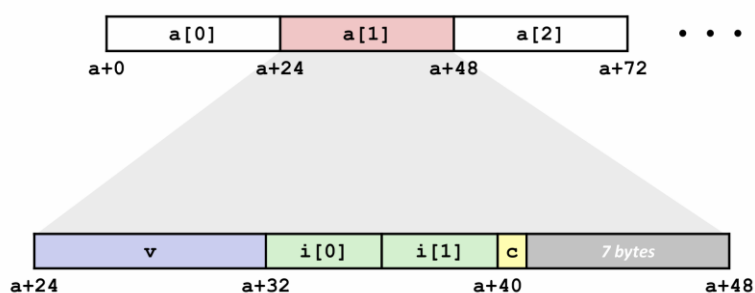
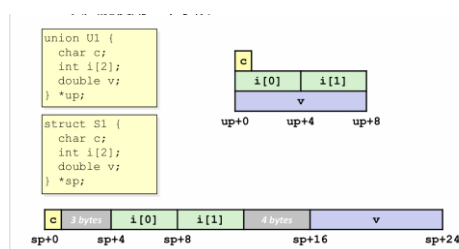
Switch 语句的跳转表：跳转表存放在只读数据段（.rodata 节），汇编里表现为间接跳转 `jmp *.L4(,%rdi,8)`。

寄存器保存约定：调用者保存（如 `%rax, %rdi, %rsi`）：被调用函数可以随便改，调用者如果还要用，得自己先压栈。

被调用者保存（如 `%rbx, %rbp, %r12-%r15`）：被调用函数用之前必须压栈，退出前恢复。

结构体与联合体对齐：结构体：每个成员的偏移量必须是其自身大小的整数倍（x86-64 下指针、double、long 都是 8 字节对齐）。整个结构体的总大小必须是最大对齐边界的整数倍。

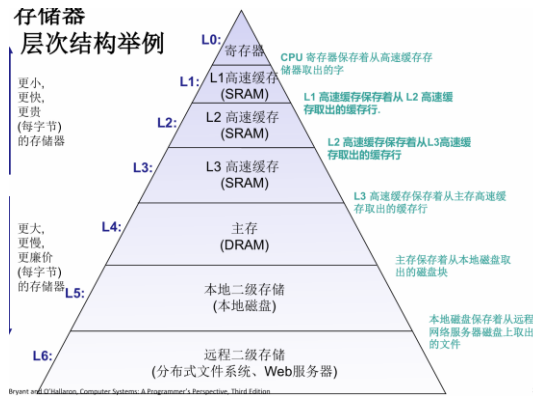
联合体：所有成员共享起始地址，总大小等于**最大成员**的大小，且同样要满足对齐。



第六章：存储器层次结构 (Cache 与磁盘)

关于存储器层次结构：

其中 RAM 易失，ROM 非易失。



1. 磁盘存储器

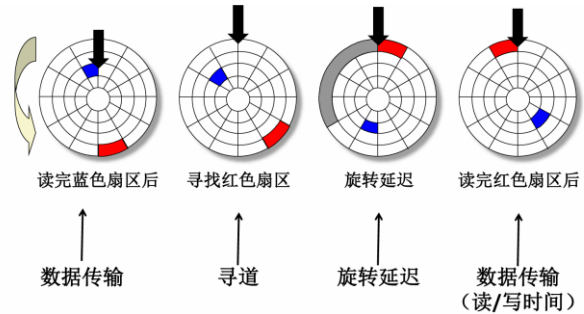
公式：磁盘容量 = (字节数/扇区) × (平均扇区数/磁道) × (磁道数/盘面) × (盘面数/盘片) × (盘片数/磁盘)

访问时间 = 寻道时间 + 旋转延迟 + 传输时间

寻道时间：磁头移动到指定磁道
(最慢，通常 3-9ms)。

旋转延迟：磁盘旋转把目标扇区转到磁头下 (平均为最大旋转延迟的一半，7200RPM 下约 4ms)。

传输时间：读写数据的时间 (极快，通常零点几 ms)。



2. 局部性原理

概念：时间局部性：被引用过一次的存储器位置，在未来可能还会被引用。(典型场景：循环、栈操作、重复调用同一函数)

空间局部性：如果一个存储器的位置被引用，那么将来它附近的位置也会被引用。(典型场景：数组顺序访问、指令顺序执行)

题：简答题，“以下代码体现了哪种局部性？”或者“如何提高 Cache 命中率？”
(增大 Cache 容量、增大块大小利用空间局部性、提高相联度减少冲突等)。

3. Cache 性能计算

公式：命中率 $h = \frac{N_c}{N_c + N_m}$ (N_c 为 Cache 访问次数， N_m 为主存访问次数)。

平均访问时间 T_a 公式： $T_a = h \times T_c + (1 - h) \times T_m$ 。假设 CPU 给出地址后 Cache 和主存同时查找，命中时 Cache 先返回。(一般来说用这个?)

对串行查找/先访 Cache 情况： $T_a = T_c + (1 - h) \times T_m$ 。假设先访问 Cache，不命中再访问主存。此时无论是否命中，都至少花费一次 T_c 的时间。

4. Cache 映射方式与地址划分

三种映射方式

直接映射：主存块只能映射到 Cache 中唯一的一个位置。($j = i \bmod C$)

全相联映射：主存块可以映射到 Cache 中的任意位置。

组相联映射：Cache 分为 2^r 组，每组有 E 行 (E 路组相联)。主存块先映射到组，再在组内全相联。

地址字段划分 假设主存地址 m 位，Cache 共有 C 行，每行 (块) B 字节。

块内偏移： $b = \log_2(B)$ 位。

组索引 : $s = \log_2(C/E)$ 位。(直接映射 $E = 1$, 全相联 $s = 0$)。

标记 : $t = m - s - b$ 位。

5. 结合代码计算命中率

逻辑: Cache 按块 (行) 加载数据。

数组在内存中是**行优先**连续存储的。

遍历顺序与存储顺序一致 (如按行遍历二维数组), 空间局部性好, 命中率高。

遍历顺序与存储顺序不一致 (如按列遍历), 每次都跨块访问, 命中率极低。

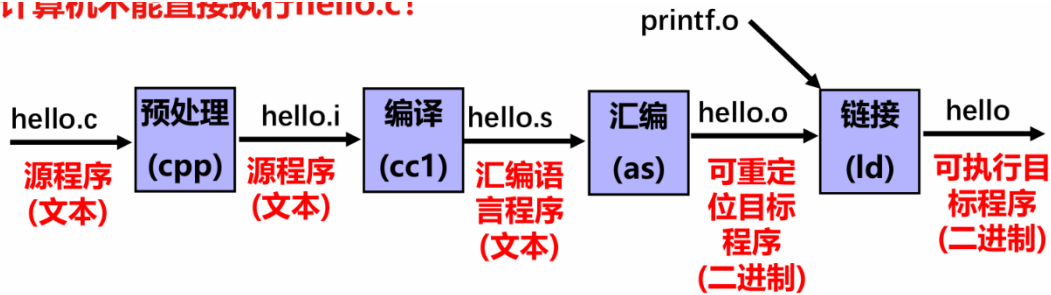
6. Cache 的写策略

写直达: 写 Cache 的同时也写主存。优点是一致性好, 缺点是慢。

写回: 只写 Cache, 加一个“脏位(Dirty bit)”, 只有当这块被替换出 Cache 时才写回主存。优点是快, 节省总线带宽。缺点是实现麻烦&容易导致错误

第七章：程序的链接

可以编译但不能直接执行hello.c:



1. 目标文件与 ELF 格式

三种目标文件

可重定位目标文件 (.o): 包含代码和数据，地址从 0 开始，不能直接执行，需要和其他.o 合并。

可执行目标文件 (a.out/可执行程序): 包含所有合并后的代码和数据，地址是虚拟内存地址，可以直接加载执行。

共享目标文件 (.so / DLL): 特殊的可重定位文件，在加载或运行时动态链接。

ELF 文件的核心/“节”

.text: 编译后的机器代码。

.rodata: 只读数据 (如 printf 的格式串、switch 语句的跳转表)。

.data: 已初始化的全局/静态变量。

.bss: 未初始化的全局/静态变量 (只是占位符，不占磁盘空间，加载时初始化为 0)。

.symtab: 符号表 (存放全局/外部符号，不包括局部变量)。

.rel.text / .rel.data: 重定位信息 (记录代码/数据中需要修改的引用地址)。

注意: 可执行文件中没有 .rel 节，因为链接时已经重定位完了; 可执行文件中也没有用户栈 (栈是运行时动态生成的)。

ELF 头
.text 节
.rodata 节
.data 节
.bss 节
.symtab 节
.rel.txt 节
.rel.data 节
.debug 节
.strtab 节
.line 节
Section header table (节头表)

2. 符号解析与强弱规则

符号分类

全局符号: 非静态的函数和全局变量 (可被其他模块引用)。

外部符号: 本模块引用，但由其他模块定义的全局符号。

本地符号: static 修饰的函数和全局变量 (仅本模块可见)。

注意: 局部非静态变量 (如函数内的 int temp) 分配在栈上，不是链接符号!

强弱符号解析规则

强符号: 函数名、已初始化的全局变量。

弱符号: 未初始化的全局变量。

规则 1: 强符号不能多次定义，否则链接错误 (ERR)。

规则 2: 一强多弱，选强 (对弱符号的引用解析为强定义)。

规则 3: 多个弱符号，任选其一 (通常按命令行扫描顺序，或编译器警告)。

3. 静态库与链接顺序

静态库 (.a) 的解析机制

链接器从左到右扫描命令行上的 .o 和 .a 文件。

维护一个未解析符号集合 U。遇到 .a 时，只提取能解析 U 中符号的 .o 模块，不需要的直接丢弃。

所以命令一般来说.a 在后，另外保证调用关系中靠前的在前面

- 假设调用关系如下：

func.o → libx.a 和 liby.a 中的函数

libx.a → liby.a 同时 liby.a → libx.a

则以下命令可行：

– gcc -static -o myfunc func.o libx.a liby.a libx.a

两种重定位类型

PC 相对引用 (如 call 指令)：目标地址相对于当前 PC 的偏移量。公式： $*refptr = ADDR(symbol) + *refptr - refaddr$ 。

绝对引用 (如指针初始化 int *p = &buf)：直接填入绝对地址。公式： $*refptr = ADDR(symbol) + *refptr$ 。

5. 动态链接 (了解即可)

共享库 (.so)：在程序加载时 (ld-linux.so) 或运行时 (dlopen) 链接。

优势：多个进程共享内存中的同一个库副本 (省内存)；磁盘上只需一份 (省空间)；库升级不需要重新编译应用程序。

PIC (位置无关代码)：共享库可以被加载到内存的任意位置。内部引用用 PC 相对寻址，外部引用通过 **GOT (全局偏移表)** 和 **PLT (过程链接表)** 实现延迟绑定。